

AWS Database Blog

Minimize downtime when migrating your Oracle database to Amazon RDS for Oracle with transportable tablespaces and AWS DMS

by Manash Kalita, Sudip Acharya, and Viqash Adwani | on 07 MAY 2024 | in [Advanced \(300\)](#), [Amazon RDS](#), [RDS for Oracle](#), [Technical How-to](#) | [Permalink](#) | [Comments](#) | [Share](#)

Organizations want to move their critical Oracle workloads to [Amazon Relational Database Service \(Amazon RDS\) for Oracle](#) with minimal downtime and disruption to unlock the agility, elasticity, and innovation of the AWS Cloud. In this post, we explore options for migrating Oracle databases from a legacy platform (for example HPUX, AIX, SOLARIS and others) to Amazon RDS for Oracle using [RMAN Cross-Platform Transportable Tablespaces Backup Sets](#) (XTTS) and [AWS Database Migration Service](#) (AWS DMS).

Oracle database migration using XTTS is a physical migration strategy where Oracle data is copied at the block level. The downtime associated with this approach is not related to database size, it's dependent on the number of metadata objects (tables and indexes) to be migrated.

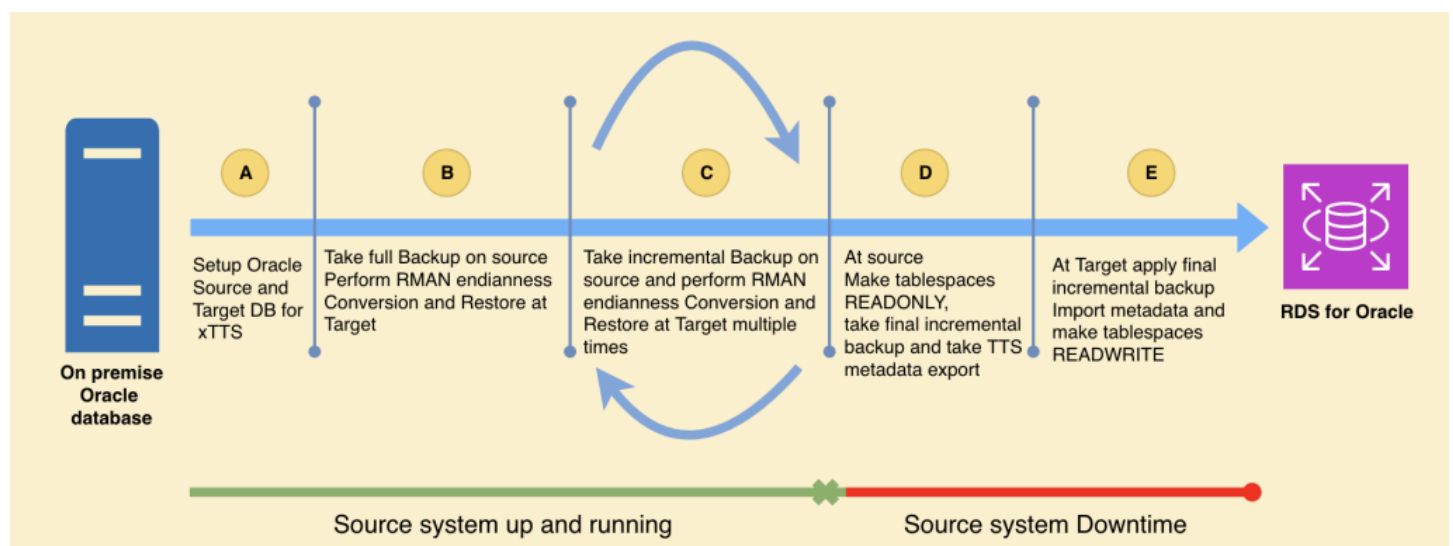
Oracle database migration using AWS DMS is a logical migration strategy where only changed data is captured from the source and applied to the target database logically. Throughout this process, the source system is available for the application workload. Only a brief downtime is needed to switch over applications to the target RDS for Oracle database.

In general, physical migration methods are more efficient than logical migration for large volumes of data, but they need more downtime than logical migration.

The solution we propose combines the two approaches: RMAN Cross-Platform Transportable Tablespace Backup Sets for full load, and AWS DMS for incremental data sync to minimize downtime.

Solution overview

As shown in the following figure, by using the XTTS method only, you minimize downtime substantially because the source database can be open read/write until the final incremental backup that requires tablespace to be in read-only mode (Step D).

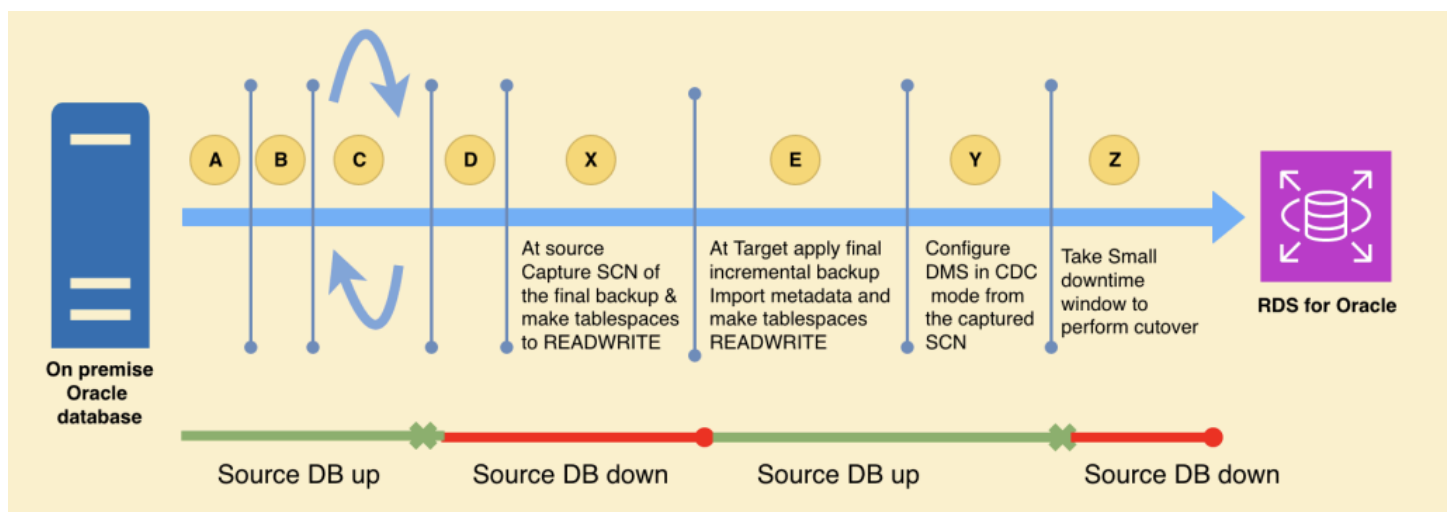


The following are the steps to migrate using XTTS only:

- (A) Set up your Oracle source and target RDS for Oracle database.
- (B) Take a full backup (Level =0) on the source and restore it on the target.
- (C) Take an incremental backup on the source and restore it on the target. This will be a repeatable process until you are ready to put the tablespace in read-only mode on the source database.
- (D) Make the tablespace read-only on the source, take a final incremental backup, and complete a metadata export via Oracle Data Pump
- (E) Restore final incremental backup on the target and import metadata objects using Oracle Data Pump. Make the tablespace read/write on the target RDS for Oracle database.

During the downtime window (Steps D and E), we observed more time consumed by the metadata import phase (Step E). Because the metadata import is a single-threaded operation, the import time is directly proportional to the number of metadata objects.

You can further minimize downtime by combining AWS DMS with the XTTS method, as shown in the following figure. In this approach, on the source database, we bring back tablespaces to read/write mode just after the export is complete. From that point of time onwards, we migrate the data using AWS DMS.



The steps to migrate using XTTS and AWS DMS are as follows (Steps A to D and Step E are same as the previous architecture):

- (A) Set up the Oracle source and target RDS for Oracle databases.
- (B) Take a full backup (Level =0) on the source and restore it on the target.
- (C) Take an incremental backup on the source and restore it on target. This will be a repeatable process until you are ready to put tablespace in read-only mode on the source database.
- (D) Make the tablespace read-only on the source, take a final incremental backup, and complete a metadata export via Oracle Data Pump.
- (X) Capture the SCN from the source database, which is required to replicate ongoing changes via AWS DMS. Change the tablespace on the source to read/write mode.

- (E) Restore the final incremental backup on the target and import metadata objects using Oracle Data Pump. Change the status of the tablespace to read/write on the target RDS for Oracle database.
- (Y) Configure AWS DMS to replicate ongoing changes from the SCN recorded in Step X.
- (Z) Complete a small outage to perform the production database cutover to Amazon RDS for Oracle.

This post [Amazon RDS for Oracle Transportable Tablespaces using RMAN](#) provides all details for setup, configuration, and migration to Amazon RDS for Oracle via transportable tablespaces using RMAN. We don't cover the same details in this post; instead, we focus on how you can integrate AWS DMS to make near-zero downtime migration possible.

The following steps were carried out for testing purposes, therefore the tablespace and table names are set accordingly. For actual database migration, you must update these per your existing source database environment.

Prerequisites

Before you start migration, you must meet the following prerequisites:

- Have a source Oracle database running on an existing on-premises or [Amazon Elastic Compute Cloud](#) (Amazon EC2) environment.
- Have a target RDS for Oracle database.
- Have [Amazon Elastic File System](#) (Amazon EFS) mounted to the source database as well as the target Oracle RDS instance. Refer to [Amazon EFS integration](#) for more details.
- [Set up](#) Oracle as the source database for change data capture (CDC) using AWS DMS.
- List the data files on the source database and take a note of them.
- Make sure the target database has the same character set as the source database.
- If there are any objects in the SYSTEM tablespace, move them to a different tablespace for TTS.
- Identify the tablespaces to be transported to the target database (specifically, any tablespaces with actual data objects that will be required for migration).
- Make sure the selected tablespaces are self-contained and TDE is not enabled.
- Create the schemas required for the desired transportable tablespace in the destination database without objects.

For this post, we use the following infrastructure:

- **Source database** – 19c on EC2 (Oracle Linux – r5.2xlarge)
- **Target database** – RDS for Oracle 19c (db.r5.2xlarge)

Setting up the source database

Complete the following steps to prepare your source database:

1. Create a tablespace that will later be migrated using XTTS:

```
SQL> create tablespace XTTDMS datafile '/u01/oradata/ORCLOEM/xttdms_1.dbf' size 1
```

2. Make sure the database user has permissions to write to the tablespace:

```
SQL> alter user dms_sample quota unlimited on XTTDMS;
```

3. Create a table in the tablespace:

```
SQL> create table dms_sample.tablEXTTS (a int primary key, b varchar2(20)) tables
```

4. Insert a few records in the table:

```
SQL> insert into dms_sample.tablEXTTS values (1, 'Before_Level-0');
```

Take a full backup on the source database

Complete the following steps to take a Level 0 transportable tablespace backup from the source database using XTTS:

1. Update the tablespace name in the `xtts.properties` file:

```
#linux system
platformid=13
#list of tablespaces to transport
tablespaces= XTTDMS
#location where backup will be generated
src_scratch_location=/mnt/efs/fs1/datapump1/xtt/xtts_scratch
#RMAN command for performing backup
#This should be set if using 12c or higher.
usermantransport=1
```

Initial a Level 0 backup using the xtts script:

```
cd /mnt/efs/fs1/datapump1/xtt
export TMPDIR=/mnt/efs/fs1/datapump1/xtt/xtts_tmp
$ORACLE_HOME/perl/bin/perl xttdriver.pl --backup --debug 3
```

The script checks itself if any previous backups are available for these tablespaces. If not, it automatically initiates a `Level 0` backup. This will be recorded in the log file as well. The output of script looks like the following code:

```
Recovery Manager: Release 19.0.0.0.0 - Production on Mon Dec 4 01:06:54 2023
Version 19.3.0.0.0
Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights reserved.
RMAN-06005: connected to target database: ORCLOEM (DBID=3703663678)
RMAN> #PLAN:XTTOMS:::13429769
2> backup for transport allow inconsistent incremental level 0 datafile
3> #NEWDESTDF:14,/XTTOMS_14.dbf
4> 14
5> #PLAN:14
6> format '/mnt/efs/fs1/datapump1/xtt/xtts_scratch/%N_%f_%U.bkp';
7>

RMAN-03090: Starting backup at 04-DEC-23
RMAN-06009: using target database control file instead of recovery catalog
RMAN-08030: allocated channel: ORA_DISK_1
RMAN-08500: channel ORA_DISK_1: SID=383 device type=DISK
RMAN-08048: channel ORA_DISK_1: starting incremental level 0 datafile backup set
RMAN-08010: channel ORA_DISK_1: specifying datafile(s) in backup set
```

The backup script also generates a file called `res.txt` in the xtts temp folder that notes the name of Level 0 backup file and SCN. Res.txt looks like the following code:

```
#0:::14,13,XTTOMS_14_1c2d57dg_1_1.bkp,0,13429769,0,0,0,XTTOMS,XTTOMS_14.dbf
```

- Copy `res.txt` to the same location (`/mnt/efs/fs1/datapump1/xtt/xtts_scratch/`) where the backup file is generated:

```
cp /mnt/efs/fs1/datapump1/xtt/xtts_tmp/res.txt /mnt/efs/fs1/datapump1/xtt/xtts_sc
```

- Make sure the permissions are set to 777 for the backup file as well as the `res.txt` file:

```
cd /mnt/efs/fs1/datapump1/xtt/xtts_scratch chmod 777 *
```

The backup is taken on a folder location that was created on Amazon EFS. The same EFS file system is already mounted to Amazon RDS for Oracle, so you don't need to copy the backup file. This reduces the overhead of copying the files from the source to target database server.

Restore the backup to the target database

Complete the following steps to restore the backup to the target RDS for Oracle database using XTTS:

1. Update tablespace name and run the [script](#) on the target RDS for Oracle instance to restore the Level 0 backup:

```
VAR task_id CLOB
BEGIN :task_id:=rdsadmin.rdsadmin_transport_util.import_xtts_tablespaces('XTTDMs'
END;
/

PRINT task_id
```

2. Note down the `TASK_ID`.
3. Check the status of task to make sure there were no errors when running the restore command on the target RDS for Oracle database:

```
SELECT * FROM rdsadmin.rds_xtts_operation_info where xtts_job_name = '17016521581
```

A successful completion of the restore command shows the column `XTTS_OPERATION_STATE` as `COMPLETED`. In the event of any errors, the column will be updated as `FAILED`.

To identify the reason of failure, a file (`FAILED`) will be created in the same folder as the physical file path on Amazon EFS. You need to look at the reason for the failure and fix it before running the `restore` command again.

You won't be able to see any tablespace or data on the target RDS for Oracle database until the metadata backup is restored. At this stage, the Level 0 backup is successfully restored on the target RDS for Oracle database.

4. Insert a few records before starting the Level 1 backup. Or you can continue with your normal operations on the source database until you're ready to start the next incremental Level 1 backup.

```
SQL> insert into dms_sample.tablEXTTS values (2,'Before_Level-1');
```

Take an incremental backup on the source database

You need to run the same XTTS perl script, which automatically detects the Level 0 backup (already taken) and initiates a Level 1 backup. At this point, you can take as many Level 1 backups as you want, and XTTS will

validate that the data is consistent. Make sure the latest `res.txt` file is copied to the target EFS directory after each incremental backup.

Restore the incremental backup to the target database

Complete the following steps to restore the Level 1 backup to the target RDS for Oracle database using XTTS:

1. Update the tablespace name and run the script on the target RDS for Oracle instance to restore the Level 0 backup:

```
VAR task_id CLOB
BEGIN :task_id:=rdsadmin.rdsadmin_transport_util.import_xtts_tablespaces('XTTOMS'
END;
/
PRINT task_id
```

2. Note down the `TASK_ID`.
3. Check the status of the task to make sure no errors occurred when running the restore command on the target RDS for Oracle database:

```
SELECT * FROM rdsadmin.rds_xtts_operation_info where xtts_job_name = '17016526403
```

The same backup folder (`/mnt/efs/fs1/datapump1/xtt/xtts_scratch`) will have a file called `xttnewdatafiles.txt` that records the location of data file that will be created in Amazon RDS for Oracle. The content of this file looks like the following code:

```
14,/rdsdbdata/db/ORCL_A/datafile//XTTOMS_14.dbf
```

It will have information regarding all the data files of tablespaces that were included in the `xtts.properties` file.

4. Copy this file to the same location as the metadata export backup folder:

```
cd /mnt/efs/fs1/datapump1/xtt/xtts_scratch
cp xttnewdatafiles.txt /mnt/efs/fs1/datapump1/
```

This is required only if the metadata backup and XTTS backup folders are different. If you're creating the metadata backup file in the same folder as the physical backup file, this step isn't required.

5. Make sure the metadata backup file and `xttnewdatafiles.txt` file have `777` permissions:

```
cd /mnt/efs/fs1/datapump1/  
chmod 777 xttdump.dmp  
chmod 777 xttnewdatafiles.txt
```

Restore the final backup on the target database

Downtime starts here. Complete the following steps to restore the final Level 1 backup on the target RDS for Oracle database using XTTS:

1. Put the tablespaces (to be migrated) in read-only mode:

```
SQL> alter tablespace XTTDMS read only;
```

2. Run the script to take a Level 1 backup:

```
cd /mnt/efs/fs1/datapump1/xtt  
export TMPDIR=/mnt/efs/fs1/datapump1/xtt/xtts_tmp  
$ORACLE_HOME/perl/bin/perl xttdriver.pl --backup --debug 3
```

The script detects a Level 0 backup and only starts an incremental backup. A successful run of the backup script looks like following code:

```
Recovery Manager: Release 19.0.0.0.0 - Production on Mon Dec 4 01:12:08 2023  
  
Version 19.3.0.0.0  
  
Copyright (c) 1982, 2019, Oracle and/or its affiliates. All rights reserved.  
  
RMAN-06005: connected to target database: ORCLOEM (DBID=3703663678)  
RMAN> set nocfau;  
2> host 'echo ts::XTTDMS';  
3> backup incremental from scn 13429769  
4> tablespace 'XTTDMS' format  
5> '/mnt/efs/fs1/datapump1/xtt/xtts_scratch/%U';  
6>  
RMAN-03023: executing command: SET NOCFAU  
RMAN-06009: using target database control file instead of recovery catalog
```

The script starts the incremental backup from the SCN that was recorded in the last successful backup:

```

/ as sysdba
size of tablespace 8
No. of tablespaces per batch 1
TABLESPACE STRING : 'XTDMS'
Prepare newscn for Tablespaces: 'XTDMS'
DECLARE*
ERROR at line 1:
ORA-20001: TABLESPACE(S) IS READONLY OR,
OFFLINE JUST CONVERT, COPY
ORA-06512: at line 284

#####
Warning:
-----
Warnings found in executing /mnt/efs/fs1/datapump1/xtt/xtts_tmp/backup_Dec4_Mon.
#####
DECLARE*
ERROR at line 1:

```

There will be a warning in the last incremental backup because the tablespace was in read-only mode. That is expected to confirm that the data is consistent.

The backup script appends the Level 1 backup information in the same res.txt file and a new line will be added in the same file with the file name generated by Level 1 as well as the new SCN:

```

#0:::14,13,XTDMS_14_1c2d57dg_1_1.bkp,0,13429769,0,0,0,XTDMS,XTDMS_14.dbf
#1:::14,13,1e2d57nb_1_1,13429769,13431017,0,0,0,XTDMS_14.dbf,XTDMS_14.dbf

```

- Copy `res.txt` to the same location (`/mnt/efs/fs1/datapump1/xtt/xtts_scratch/`) where the backup file is generated:

```

cp /mnt/efs/fs1/datapump1/xtt/xtts_tmp/res.txt /mnt/efs/fs1/datapump1/xtt/xtts_sc

```

- Make sure the permissions are set to `777` for the backup file as well as the `res.txt` file:

```

cd /mnt/efs/fs1/datapump1/xtt/xtts_scratch
chmod 777 *

```

- Record the SCN. This is required for AWS DMS to start replicating changes only from that point to the target RDS for Oracle instance.

```
more /mnt/efs/fs1/datapump1/xtt/xtts_tmp/xttplan.txt
XTTDMS:::13431017
14
```

6. Export the metadata backup from the source database:

```
expdp system dumpfile=xttdump.dmp directory=META_DIR statistics=NONE
transport_tablespaces=XTTDMS transport_full_check=y logfile=tts_export.log
```

The Data Pump backup is also taken on a directory that is mounted on Amazon EFS. Because Amazon RDS for Oracle has the same EFS directory, there is no need to copy the backup files from source to target database server.

7. Put the tablespace in read/write mode:

```
alter tablespace XTTDMS read write;
```

Downtime ends at this stage. The source database is now up and running and available for operations from the application from any user. If you're not using AWS DMS, downtime will continue until the Level 1 backup is restored and the Data Pump metadata import is complete.

8. You can resume normal database operations on the source database at this time. All records at this stage won't be available in the Level 1 backup and will be managed by AWS DMS only.

```
SQL> insert into dms_sample.tablextts values (3, 'After_Level-1');
SQL> insert into dms_sample.tablextts values (4, 'Before_DMS');
```

Import the transportable tablespace metadata

Complete the following steps to import the metadata objects:

1. Restore the metadata backup on the RDS for Oracle instance:

```
exec rdsadmin.rdsadmin_transport_util.import_xtts_metadata('xttdump.dmp', 'DATA_PU
```

This command takes time to complete based on the number of objects for which the metadata backup needs to be restored. When it's complete, you should see a PL/SQL procedure successfully completed message. However, that doesn't necessarily mean that the job completed successfully.

2. To verify the import was completed successfully and there were no errors, you can verify the contents of the metadata Data Pump import log:

```
SELECT * FROM TABLE
(rdsadmin.rds_file_util.read_text_file(
p_directory => 'DATA_PUMP_DIR_EFS',
p_filename => 'tts_export.log'));
```

After the successful completion of the metadata import, you should see the tablespace on the target RDS for Oracle database in read-only mode. The tablespace is the consistent image of the backup that was taken from the source database. During that period, the tablespace was also in read-only mode in the source database.

3. To confirm that the tablespace on the target database has the same data as the source database, put the tablespace in read/write mode in the target RDS for Oracle instance:

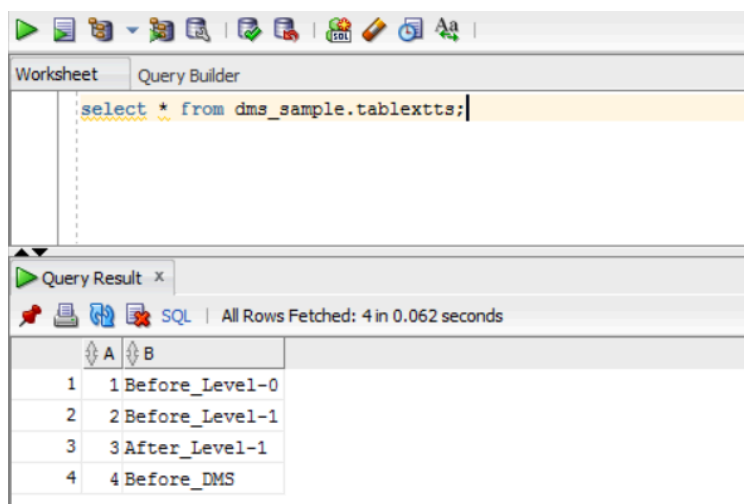
```
alter tablespace XTTDMS read write;
```

Replicate changes using AWS DMS

At this point, your target database has the copy of data from the source that was taken before putting the tablespace in read-only mode. All the transactions that happened after that will be missing from the target database and will be replicated by AWS DMS.

For example, in our test environment, the data in the source and target database looked like the following screenshots.

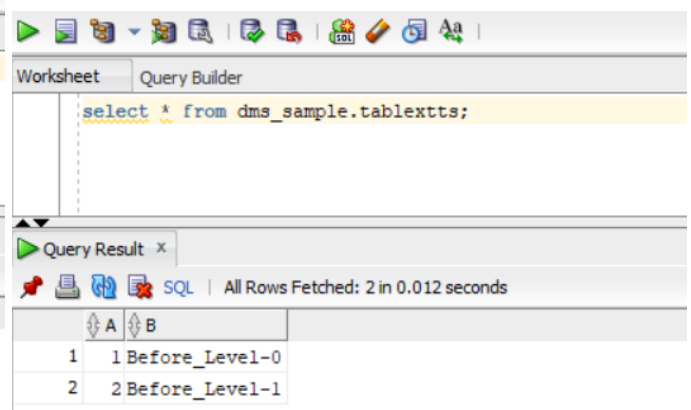
Source Database Table



The screenshot shows a SQL query result in a 'Query Result' window. The query is 'select * from dms_sample.tablextts;'. The result shows 4 rows fetched in 0.062 seconds. The data is as follows:

	A	B
1	1	Before_Level-0
2	2	Before_Level-1
3	3	After_Level-1
4	4	Before_DMS

Target Database Table



The screenshot shows a SQL query result in a 'Query Result' window. The query is 'select * from dms_sample.tablextts;'. The result shows 2 rows fetched in 0.012 seconds. The data is as follows:

	A	B
1	1	Before_Level-0
2	2	Before_Level-1

Looking at this data, we can verify that the target database only has records that were inserted before the Level 1 backup on the source database. All the records after that are missing and need to be replicated.

Because AWS DMS enables you to start [replicating changes from a CDC start point](#), you can use it to replicate ongoing changes. You have to use the SCN that you recorded as part of the final Level 1 backup from the source database.

To start replication using AWS DMS, complete the following steps:

1. [Create a replication instance.](#)
2. [Create source and target endpoints.](#)
3. Make sure that the AWS DMS replication instance network is [set up correctly](#) and is able to connect to your source and target database.
4. [Create the AWS DMS replication task](#), which replicates ongoing changes. The following are important settings in the task creation process:
 - a. For **Migration type**, select **Replicate data changes only**. This will only replicate changes, leaving the existing data untouched on the target database.
 - b. Select **Enable custom CDC start mode** to make sure CDC changes are replicated from a specific SCN.
 - c. For **System Change Number**, enter the SCN you recorded earlier. To make sure data consistency is maintained, you use the SCN that was generated after the Level 1 backup.
 - d. For **Target table preparation mode**, choose **Do nothing**. This confirms that tables on the target database will remain as is and only data will be replicated.
 - e. For **Table mappings**, specify the schema name and table names that you want AWS DMS to replicate ongoing changes.
 - f. Choose **Restart** to make sure that changes are started from the beginning of the SCN you specified.

After the task is restarted, it will start replicating changes, and the status of the task will show as **Replication ongoing**.

Identifier	Status	Progress	Type	Source	Target	Replication instance	Started	Stopped	Elapsed load time	Tables loaded	Tables loading	Tables queued
xttsdmsorastask	Replication ongoing	100%	Ongoing replication	oracle-source	oracle-target	repinstance-351	December 4, 2023 at 12:27:01 (UTC+11:00)	December 4, 2023 at 10:47:41 (UTC+11:00)	4 s	1	0	0

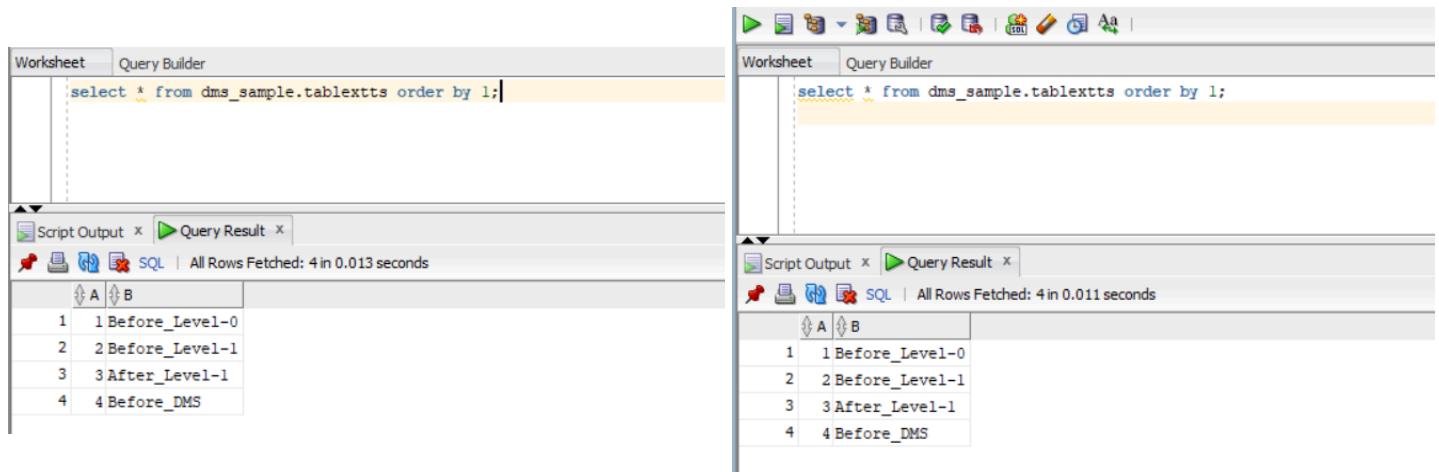
You can open the task for more information and confirm if you are able to see the changes being replicated on the target database.

Schema name	Table	Load state	Elapsed load time	Inserts	Deletes	Updates	DOLs	Applied inserts	Applied deletes	Applied updates	Applied DOLs	Full load rows	Total rows	Validation state	Validation pending	Validated
DMS_SAMPLE	TAB1EXT1S	Table completed	2 s	2	0	0	0	2	0	0	0	0	2	Not enabled	0	0

Finally, you can compare the records on the target database to confirm that it's in sync with the source database, as shown in the following screenshots.

Source Database

Target Database



The image displays two side-by-side screenshots of the AWS DMS console. Both screenshots show the 'Query Builder' and 'Query Result' tabs. The 'Query Builder' tab contains the SQL query: `select * from dms_sample.tablextts order by 1;`. The 'Query Result' tab shows the results of the query, which are 4 rows of data: '1 Before_Level-0', '2 Before_Level-1', '3 After_Level-1', and '4 Before_DMS'.

Clean up

To avoid incurring future charges, complete the following steps:

1. [Stop the EC2 instance](#) (if your source Oracle database was deployed on one).
2. [Delete the RDS for Oracle instance](#).
3. [Delete the EFS file system](#).
4. Delete all AWS DMS replication tasks you created before you [delete the AWS DMS replication instance](#).

Conclusion

In this post, we explored migration of an Oracle database to Amazon RDS for Oracle. With a combined approach of RMAN XTTS for the bulk data load and AWS DMS for ongoing changes, you can minimize your downtime requirements considerably. With careful planning, organizations can migrate their Oracle workloads to Amazon RDS for Oracle and take advantage of the flexibility, scalability, and innovation of AWS. This migration option enables you to modernize your Oracle environment on your own timeline.

Try this approach in your next Oracle database migration to Amazon RDS for Oracle. If you have any comments or questions, leave them in the comments section.

About the Authors



Viqash Adwani is a Sr. Database Specialty Architect with Amazon Web Services. He works with internal and external Amazon customers to build secure, scalable, and resilient architectures in the AWS Cloud and help customers perform migrations from on-premises databases to Amazon RDS and Amazon Aurora databases.

Sudip Acharya is a Sr. Consultant with the AWS ProServe team in India. He works with internal and external Amazon customers to provide guidance and technical assistance on database projects, helping them improve the value of their solutions when using AWS.



Manash Kalita is a Database Specialist Solutions Architect with Amazon Web Services. He works with AWS customers designing customer solutions on database projects, helping them migrate and modernize their existing databases to the AWS Cloud as well as orchestrate large-scale migrations in AWS.

Comments

0 Comments

 Login ▼

G

Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS 

Name



Share

Best Newest Oldest

Be the first to comment.

Subscribe

Privacy

Do Not Sell My Data